

CSE222 / BİL505
Data Structures and Algorithms
Homework #6 – Report

Sümeyye Zeynep Gürbüz

- **Selection Sort**

Time Analysis	Time complexity of this sort is $O(n^2)$. It doesn't matter if the array is sorted, randomized, or reversely sorted. It always makes 28 comparisons and 7 swaps. This actually means that this sort is really unefficient in terms of time since the best-case requires the same amount of time with the worst-case.
Space Analysis	Space complexity of this sort is $O(n)$ since all variables except the array itself requires constant space.

- **Bubble Sort**

Time Analysis	Time complexity of this sort is $O(n^2)$ in the worst-case (reversely sorted array), and $O(n)$ in the best-case (sorted array). When the array is randomized, it makes 27 comparisons - which is only one short than the worst-case scenario; but there is a notable difference between swap amounts since it requires almost half as much swaps as the worst-case scenario.
Space Analysis	Space complexity of this sort is $O(n)$ since all variables except the array itself requires constant space.

- **Quick Sort**

Time Analysis	This sort has two functions, one of them is recursive: 1) Time complexity of the partitioning part is $O(n)$ for all cases. 2) Time complexity of the recursive sorting part is $O(n^2)$ for the worst-case (reversely sorted array), and $O(n \log n)$ for both randomized and sorted arrays. If comparison amounts are considered, it can be seen that for both the best (ordered array) and the worst (reversely ordered array) cases, it requires the same amount; for a randomized array, it requires half amount of comparisons. But if swap amounts are considered, it can be seen that the best-case (10 swaps) occurs when the array is randomized, and the worst-case (35 swaps) occurs when the array is already sorted.
Space Analysis	Space complexity of the partitioning part is $O(1)$ for all cases; and space complexity of the sorting part is $O(n)$ for the worst-case (reversely ordered array), and $O(\log n)$ for randomized or sorted arrays.

- **Merge Sort**

Time Analysis	This sort has two functions, one of them is recursive: 1) Time complexity of the merging part is $O(n)$ for all cases. 2) Time complexity of the recursive sorting part is $O(n \log n)$ for all cases. This sort needs no swap, and needs approximately 10 to 15 comparisons for all cases (sorted, randomized, reversely sorted).
Space Analysis	Space complexity of the merging part is $O(n)$ due to the temporary arrays, and space complexity of the recursive sorting part is $O(\log n)$ due to the function call stack.

General Comparison of the Algorithms

In summary, it can be said that different sorting algorithms are used for different situations. It depends on the programmer's requirements and the system's capabilities.

For this scenario, where we have three different arrays of eight elements - a sorted, a randomized and a reversely sorted - Merge Sort and Quick Sort showed better performance in terms of time and space complexity in comparison with Selection Sort and Bubble Sort.

If we inspect the selection and bubble sort, we can see that they almost have the same amounts of comparisons and swaps in general. This means that they did the same amount of work for all cases, without remark to the input. This situation makes these algorithms relatively least efficient for time and space usages.

Even though quick sort looks like having the same numbers with the selection and bubble sort for the comparison and swap operations, it is actually efficient for average-cases, and since the real life systems most likely has randomized inputs, we can say that the quick sort can be more efficient than the selection and bubble sort.

Finally, when we analyze the merge sort, it is obvious that this is the most efficient algorithm nearly for all cases. As it does not require any swaps, comparison amounts are almost half of other algorithms.